

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.V.N.S.BHIMESWAR	Reg. No.:	192472048
Section / Batch:	B.Tech	Date:	16/07/2026

Code of Conduct

I __ C.Jaswanth (192424334) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____ C.Jaswanth ____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

ANSWER:- Aim

To design and implement a Python program using **Regular Expressions (Regex)** to extract candidate information from a resume, including name, email, mobile number, technical skills, and years of experience, and determine whether the candidate satisfies the eligibility criteria.

Algorithm

1. Import the re module.
2. Read the resume text.
3. Use Regular Expressions to:
 - Extract the candidate's name.
 - Find the email address.
 - Find the mobile number.
 - Find the years of experience.
4. Search for technical skills from the given list:
 - Python
 - Java
 - SQL
 - Machine Learning
 - NLP
5. Store the extracted information.
6. Display the structured candidate profile.
7. Check eligibility:
 - Experience ≥ 2 years.
 - Python skill is present.
8. If both conditions are satisfied, display **Eligible**; otherwise display **Not Eligible**.
9. Stop.

Python Code

```
import re

resume = """
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
"""

# Name
name = re.search(r"Name:\s*(.*)", resume)

# Email
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)

# Mobile
```

```

mobile = re.search(r"\b\d{10}\b", resume)

# Experience
exp = re.search(r"(\d+)\s+years", resume)

# Skills
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = []

for skill in skill_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills.append(skill)

experience = int(exp.group(1))

print("----- Candidate Profile -----")
print("Name :", name.group(1))
print("Email :", email.group())
print("Mobile :", mobile.group())
print("Skills :", " , ".join(skills))
print("Experience :", experience, "years")

if experience >= 2 and "Python" in skills:
    print("Status : Eligible")
else:
    print("Status : Not Eligible")

```

Sample Input

Name: Rahul Sharma
 Email: rahul@gmail.com
 Mobile: 9876543210
 Skills: Python, Java, SQL, Machine Learning
 Experience: 3 years

Sample Output

----- Candidate Profile -----
 Name : Rahul Sharma
 Email : rahul@gmail.com
 Mobile : 9876543210
 Skills : Python, Java, SQL, Machine Learning
 Experience : 3 years
 Status : Eligible

Side Heading 1: Extract the Candidate's Name

Regex Used

`r"Name:\s*(.*)"`

Explanation

- Searches for the word **Name**:
- `\s*` matches spaces.
- `(.*)"` extracts the candidate's name.

Output

Rahul Sharma

Side Heading 2: Identify Email Addresses

Regex Used

`r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"`

Explanation

- Finds a valid email format.
- Matches username, @, domain name, and extension.

Output

rahul@gmail.com

Side Heading 3: Extract Mobile Numbers

Regex Used

```
r"\b\d{10}\b"
```

Explanation

- \d{10} matches exactly 10 digits.
- \b ensures it is a complete mobile number.

Output

```
9876543210
```

Side Heading 4: Detect Technical Skills**Skills Checked**

- Python
- Java
- SQL
- Machine Learning
- NLP

Code

```
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
```

```
skills = []
```

```
for skill in skill_list:
```

```
    if re.search(skill, resume, re.IGNORECASE):
```

```
        skills.append(skill)
```

Output

```
Python
```

```
Java
```

```
SQL
```

```
Machine Learning
```

Side Heading 5: Extract Years of Experience**Regex Used**

```
r"(\d+)\s+years"
```

Explanation

- \d+ matches one or more digits.
- Extracts the number before the word **years**.

Output

```
3 years
```

Side Heading 6: Generate Structured Summary**Summary**

```
----- Candidate Profile -----
```

```
Name : Rahul Sharma
```

```
Email : rahul@gmail.com
```

```
Mobile : 9876543210
```

```
Skills : Python, Java, SQL, Machine Learning
```

```
Experience : 3 years
```

Side Heading 7: Display Eligible Candidates**Eligibility Criteria**

- Experience ≥ 2 years
- Must have **Python** skill

Code

```
if experience >= 2 and "Python" in skills:
```

```
    print("Status : Eligible")
```

```
else:
```

```
    print("Status : Not Eligible")
```

Output

```
Status : Eligible
```

Result

The Python program successfully extracts candidate details using Regular Expressions, generates a structured profile, and identifies whether the candidate satisfies the required eligibility criteria based on experience and Python skill.

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

ANSWER:-

Aim

To develop a Python program using **Regular Expressions** to search product names based on exact keywords, prefixes, suffixes, partial keywords, and case-insensitive matching, and generate a report of matching products.

Algorithm

1. Import the re module.
2. Store product names in a list.
3. Accept the search keyword.
4. Perform:
 - o Exact keyword search.
 - o Prefix search.
 - o Suffix search.
 - o Partial keyword search.
 - o Case-insensitive search.
5. Display all matching products.
6. Count the matching products for each search.
7. Display the search report.
8. Stop.

Python Code

```
import re
```

```
products = [  
    "Laptop",  
    "Laptop Bag",  
    "Mouse",  
    "Wireless Mouse",  
    "Keyboard",  
    "Gaming Keyboard",  
    "USB Cable",  
    "Mobile Charger"  
]
```

```
keyword = "Laptop"
```

```
exact = [p for p in products if re.fullmatch(keyword, p)]  
prefix = [p for p in products if re.match(keyword, p)]  
suffix = [p for p in products if re.search(keyword + r"$", p)]  
partial = [p for p in products if re.search(keyword, p)]  
case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
```

```
print("Exact Search :", exact)  
print("Prefix Search :", prefix)  
print("Suffix Search :", suffix)  
print("Partial Search :", partial)  
print("Case-Insensitive Search :", case)
```

```
print("\nSearch Report")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))
print("Partial Matches :", len(partial))
print("Case-Insensitive Matches :", len(case))
```

Sample Input

Keyword : Laptop

Sample Output

Exact Search : ['Laptop']
Prefix Search : ['Laptop', 'Laptop Bag']
Suffix Search : ['Laptop']
Partial Search : ['Laptop', 'Laptop Bag']
Case-Insensitive Search : ['Laptop', 'Laptop Bag']

Search Report
Exact Matches : 1
Prefix Matches : 2
Suffix Matches : 1
Partial Matches : 2
Case-Insensitive Matches : 2

Side Heading 1: Search Products Using an Exact Keyword

Regex

```
re.fullmatch(keyword, product)
```

Explanation

Matches the complete product name exactly.

Output

Laptop

Side Heading 2: Perform Prefix-Based Searches

Regex

```
re.match(keyword, product)
```

Explanation

Matches products that start with the keyword.

Output

Laptop
Laptop Bag

Side Heading 3: Perform Suffix-Based Searches

Regex

```
re.search(keyword + r"$", product)
```

Explanation

Matches products ending with the keyword.

Output

Laptop

Side Heading 4: Search Using Partial Keywords

Regex

```
re.search(keyword, product)
```

Explanation

Finds the keyword anywhere in the product name.

Output

Laptop
Laptop Bag

Side Heading 5: Perform Case-Insensitive Searches

Regex

```
re.search(keyword, product, re.IGNORECASE)
```

Explanation

Searches without considering uppercase or lowercase letters.

Output

Laptop
Laptop Bag

Side Heading 6: Display All Matching Products

Output

Laptop
Laptop Bag

Side Heading 7: Generate Search Report

Output

Exact Matches : 1
Prefix Matches : 2
Suffix Matches : 1
Partial Matches : 2
Case-Insensitive Matches : 2

Result

The program successfully searches products using different Regular Expression techniques and generates a report showing the total number of matching products.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

ANSWER:- Aim

To develop a Python program using **Regular Expressions** to validate student registration details such as register number, institutional email, course code, semester, and mobile number, and generate the registration status.

Algorithm

1. Import the re module.
2. Read student details.
3. Validate:
 - Register number.
 - Institutional email.
 - Course code.
 - Semester.
 - Mobile number.
4. Display validation results.
5. If all fields are valid, display **Registration Successful**.
6. Otherwise, display **Registration Failed**.
7. Stop.

Python Code

```
import re

reg_no = "23CS001"
email = "student@klu.ac.in"
course = "CS301"
semester = "5"
```

```
mobile = "9876543210"
```

```
reg = bool(re.fullmatch(r"\d{2}[A-Z]{2}\d{3}", reg_no))
mail = bool(re.fullmatch(r"[a-zA-Z0-9._%+-]+@klu.ac.in", email))
code = bool(re.fullmatch(r"[A-Z]{2}\d{3}", course))
sem = bool(re.fullmatch(r"[1-8]", semester))
mob = bool(re.fullmatch(r"\d{10}", mobile))
```

```
print("Register Number :", "Valid" if reg else "Invalid")
print("Email :", "Valid" if mail else "Invalid")
print("Course Code :", "Valid" if code else "Invalid")
print("Semester :", "Valid" if sem else "Invalid")
print("Mobile Number :", "Valid" if mob else "Invalid")
```

```
if reg and mail and code and sem and mob:
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")
```

Sample Input

Register Number : 23CS001
Email : student@klu.ac.in
Course Code : CS301
Semester : 5
Mobile : 9876543210

Sample Output

Register Number : Valid
Email : Valid
Course Code : Valid
Semester : Valid
Mobile Number : Valid

Registration Successful

Side Heading 1: Validate Student Register Number

Regex

```
r"\d{2}[A-Z]{2}\d{3}"
```

Explanation

Checks for:

- 2 digits
- 2 uppercase letters
- 3 digits

Output

Valid

Side Heading 2: Validate Institutional Email Address

Regex

```
r"[a-zA-Z0-9._%+-]+@klu.ac.in"
```

Explanation

Accepts only institutional email addresses ending with @klu.ac.in.

Output

Valid

Side Heading 3: Validate Course Code

Regex

```
r"[A-Z]{2}\d{3}"
```

Explanation

Checks for two uppercase letters followed by three digits.

Output

Valid

Side Heading 4: Validate Semester Information

Regex

```
r"[1-8]"
```

Explanation

Allows only semester numbers from **1 to 8**.

Output

Valid

Side Heading 5: Validate Student Mobile Number**Regex**

```
r"\d{10}"
```

Explanation

Checks whether the mobile number contains exactly 10 digits.

Output

Valid

Side Heading 6: Display Validation Messages**Output**

Register Number : Valid

Email : Valid

Course Code : Valid

Semester : Valid

Mobile Number : Valid

Side Heading 7: Generate Final Registration Status Report**Code**

```
if reg and mail and code and sem and mob:
```

```
    print("Registration Successful")
```

```
else:
```

```
    print("Registration Failed")
```

Output

Registration Successful

Result

The Python program successfully validates all student registration details using Regular Expressions, displays validation messages for each field, and generates the final registration status based on the validation results.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____